Auditing Conditional Low-Rank Layers: An Evidence Protocol Applied to Ephemeral FFN Cores, with Negative Results on Dynamic Capacity Allocation

Da Xu *xudayj@chinamobile.com*
China Mobile Research Institute

Abstract

Conditional low-rank architectures promise input-adaptive capacity at low storage cost, and efficiency claims for them can conflate representational, storage, and execution evidence. We present an audit methodology that keeps these effects separate, and we demonstrate it on one such architecture: gated Transformer feed-forward networks (FFNs) whose low-rank factors are modulated by a generated diagonal core. The methodology pairs matched-contrast theory with provenance-checked measurement. Its theory component yields an output-subspace error floor that no controller can overcome while the final factor remains fixed, an activation-weighted constant-core control, and a matched-schedule bound on removable descriptor transfers. Its measurement component reanalyzes archived accelerator records under an explicit evidence taxonomy and adds fresh software verification. Applying it produces a predominantly negative account of dynamic capacity allocation for the studied case. In four recorded training-seed summaries for a Qwen2.5-1.5B FFN, the generated model improves relative reconstruction MSE only marginally at matched rank and is 1.95% worse at the closest reported parameter budget; original execution reports and checkpoints are unavailable, so these are auditable archive calculations rather than independently reproduced quality measurements. A same-run archived kernel summary reports 0.8234 versus 0.8509 ms for fused and materialized execution at 128 tokens, but persistent low rank takes 0.2719 ms: the dynamic path remains 3.03× slower than the static alternative it must beat. A re-run multicore variant on a second device passes a hardened 1000-call stability gate with zero transient or persistent failures, sustaining 1.05–2.97× speedups over the fused single-core kernel from T=128 to 4096, a 26–66% kernel-latency reduction at $T \geq 512$; device-counter and trained-model evidence remain open. Fresh CPU tests of the actual causal FFN reduce forward-retained nonparameter tensor storage by 13× under checkpointing, with identical outputs and gradients in nine small synthetic fixtures. The methodology itself is the transferable contribution: every component is defined for a generic conditional low-rank layer, and the retained negative findings identify which dynamic-allocation claims the studied evidence can and cannot support.

1 Introduction

Transformer feed-forward networks contain large parameter matrices and several intermediate activations. Reducing one of these objects can change arithmetic, parameter capacity, memory transfers, and kernel launch overhead simultaneously. A speedup observed after a low-rank replacement therefore does not identify which change caused it. This distinction matters for conditional weight generation: a compact controller may add expressivity, while computing and materializing its outputs can cost more than the transfers that fusion removes.

We examine MWG-EW, a causal, token-conditioned diagonal modulation of persistent low-rank FFN factors. For a row vector x, one projection is

$$p(x) = \big((xU) \odot z(x)\big)V, \quad U \in \mathbb{R}^{d_{\text{in}} \times r}, \quad V \in \mathbb{R}^{r \times d_{\text{out}}}. \tag{1}$$

The algebra is closely related to feature modulation and hypernetworks. The systems question is whether the generated descriptor z should be stored as a global tensor or consumed immediately where it is formed. We call the latter a *descriptor no-store contract*. The factors, controller parameters, and output activations retain their own lifetimes; the contract concerns only the named descriptor and a declared implementation boundary.

This paper contributes an audit methodology for conditional low-rank layers, demonstrated end-to-end on MWG-EW. It has four components, each defined for a generic modulated projection so that it transfers beyond this architecture:

1. **Matched-contrast theory** separates changing rank from changing conditional coefficients: an output-subspace obstruction and an optimal constant-core projection control, which together bound what dynamic capacity allocation can add over a static control.

2. **Memory-interface accounting** separates parameter storage, generated cardinality, scheduled global transfers, and physical DRAM traffic, so that a reported saving can be assigned to exactly one of them.

3. **An evidence taxonomy** that grades every measurement by what it can support: source audit, compiled-kernel inspection, device-counter comparison, and archive recomputation are distinct levels, and hashes certify integrity but not execution.

4. **Pre-declared validation gates** that are specified before measurement and retained after failure, because a failed gate changes which conclusions the remaining evidence supports.

Applying the methodology to MWG-EW yields a predominantly negative result for dynamic capacity allocation in the studied setting: matched-rank conditioning gains are marginal, the closest parameter-matched comparison favors the persistent baseline, and the fused dynamic path remains slower than the static alternative. We retain this negative evidence as a first-class outcome: it is the measurement of the methodology, not a caveat on an implied benefit.

The most useful empirical conclusion is a negative result about dynamic capacity allocation. Generated modulation can approximately match a trained same-rank reconstruction baseline in the available summaries, but every strongest available comparison favors the static persistent baseline: the closest parameter-matched reconstruction, the practical kernel latency, and the absence of any validated joint quality–latency gain. Under the audit's taxonomy this is evidence that the studied conditioning mechanism does not buy deployable efficiency at matched budgets, not merely evidence that more experiments are needed. Eliminating an explicit descriptor tensor is feasible at the source level; it does not imply that the complete FFN runs faster. Both conclusions are consistent with the analysis and motivate a smaller set of decisive experiments than a broad collection of disconnected speed and quality claims.

2 Relationship to Prior Work

Conditional parameters and feature modulation. HyperNetworks generate another network's parameters (Ha et al., 2017). FiLM applies conditional feature-wise affine transformations (Perez et al., 2018); Eq. (1) is multiplicative modulation in a rank-space representation. Learned static activation scales also appear in $(IA)^3$ (Liu et al., 2022). A static diagonal scale can be absorbed into a factor, whereas a token-conditioned scale generally cannot. LoRA (Hu et al., 2022) is a low-rank update to a retained base matrix, while the studied FFN replaces each dense projection by factors. These distinctions preclude treating the present algebra as an unprecedented form of generation.

The recent Ouroboros preprint uses a hidden-state controller to modulate SVD-initialized low-rank bases within recursive Transformers (Jaber & Jaber, 2026). It is especially close algebraic prior art. Our scope is the lifetime and controlled assessment of FFN descriptors, including the limits imposed by a fixed output subspace. We make no claim of priority over input-conditioned diagonal modulation.

Table 1: Which mechanism a controlled contrast can identify.

Question	Matched contrast	Required invariant
Does rank reduction help?	Dense versus trained persistent factors	Task metric, data and optimization budget
Does conditioning help?	Dynamic versus constant-core/static control	Parameter or compute budget; causal evaluation
Does descriptor fusion help?	Same dynamic model, materialized versus fused	Checkpoint, inputs, output tolerance and timing boundary
Does recomputation help?	Eager versus checkpointed same module	Parameters, input, loss and all gradients

Mixture-of-experts and dynamic capacity allocation. Sparsely gated mixture-of-experts layers condition *which* parameters participate on the input (Shazeer et al., 2017), and Switch Transformers scale this to trillion-parameter models with top-one routing (Fedus et al., 2022). MoE and the studied modulation are two answers to the same question—input-adaptive capacity—at different granularity: MoE routes among expert FFNs whose parameters are all persistent, while the studied FFN generates coefficients within one fixed low-rank subspace and stores no per-expert weights. The audit methodology is directly applicable to MoE efficiency claims, where the same conflation arises: routing logits, dispatch, and load-balancing losses are execution costs that a parameters-only comparison hides, and expert utilization should be audited against the static dense control at matched task quality. The present archive contains no MoE measurement; we claim no empirical MoE result.

I/O-aware execution and recomputation. FlashAttention analyzes and reduces transfers between memory levels (Dao et al., 2022). FlashMHF combines dynamically weighted FFN subnetworks with online SRAM computation (Zhang et al., 2025); it is a relevant future systems and model-quality comparator. Activation checkpointing trades retained tensors for recomputation (Chen et al., 2016), and Checkmate optimizes rematerialization schedules (Jain et al., 2020). Our CPU checkpoint experiment applies this established technique to the actual MWG module. It verifies compatibility and instrumentation rather than proposing a new checkpointing algorithm.

Compression and fair comparison. Quantization methods such as GPTQ and AWQ reduce the representation cost of weights (Frantar et al., 2023; Lin et al., 2024). They remain important deployment baselines. Rank reduction, quantization, modulation, and fusion should be compared at matched task quality and reported budgets. The archived experiments here do not support numerical superiority over those methods. The output rank bound uses the classical Eckart–Young–Mirsky approximation result (Eckart & Young, 1936; Mirsky, 1960).

3 Model and Execution Contract

3.1 Causal gated FFN

Let d be the model width and m the FFN intermediate width. The dense reference, using row-vector conventions, is

$$f(x) = \big[\mathrm{SiLU}(xW_g) \odot (xW_u)\big]W_d, \quad W_g, W_u \in \mathbb{R}^{d\times m}, \quad W_d \in \mathbb{R}^{m\times d}. \tag{2}$$

In the factorized model, each W_p is replaced by Eq. (1). For token state $x_{b,t}$ the controller produces all three rank vectors:

$$h_{b,t} = \mathrm{SiLU}(x_{b,t}A + a_0), \tag{3}$$

$$(z_u, z_g, z_d)_{b,t} = \mathbf{1} + \alpha \tanh(h_{b,t}C + c_0), \tag{4}$$

$$\widetilde{f}(x) = p_d\big(\mathrm{SiLU}(p_g(x)) \odot p_u(x); z_d(x)\big). \tag{5}$$

Here $A \in \mathbb{R}^{d\times h}$ and $C \in \mathbb{R}^{h\times 3r}$; all factors and controller parameters are trainable persistent state. The down-projection core is generated from the original pre-FFN state x, while its projection input is the gated

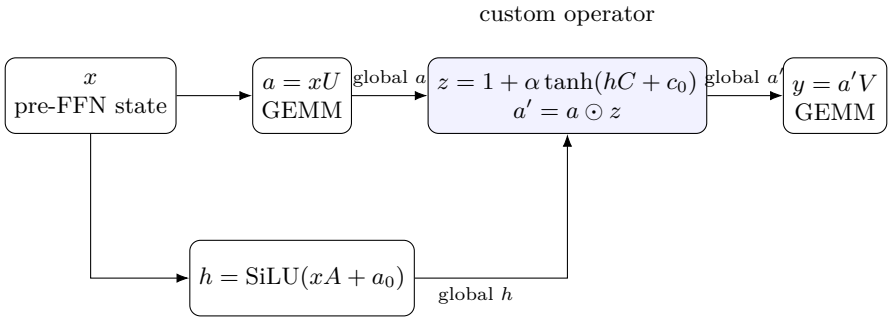


Figure 1: Implemented boundary for one projection. Only the descriptor z is consumed without an explicit global descriptor tensor. Source-level absence of a descriptor store does not prove absence of compiler or runtime spills.

intermediate. The controller has

$$P_G = (d + 1)h + (h + 1)3r \tag{6}$$

parameters, including biases. For equal fp32 factor/controller precision, total parameter bytes are $4[3r(d + m) + P_G]$. Optimizer states and saved activations are separate quantities.

The implemented causal mode uses a distinct core for each token ($q = 1$). It reads no future hidden states and pools no companion examples. If a future implementation shares a core across $q > 1$ tokens, both group boundaries and its context must be determined from information available by the first token's pre-FFN state. Pooling the complete group's teacher-forced states would violate this condition. A batch of B examples of length S has

$$n = B\lceil S/q \rceil, \qquad 1 \le q \le S, \tag{7}$$

groups under batch-isolated grouping. Cached decoding has $S = q = 1$ and $n = B$; we do not label a 128-token call as single-token autoregressive decoding.

3.2 The actual fusion boundary

The custom Ascend operator receives rank activations $a = xU$, controller hidden states h, and the controller head's parameters. It forms the head affine result, applies the bounded transform in Eq. (4), and multiplies a by the resulting scale. The descriptor is not returned as an explicit global tensor. The operator *does* write $a \odot z$ to global memory, and the U and V GEMMs run outside it. Consequently, neither the entire projection nor the entire FFN is scratch-resident.

We distinguish three verification levels. A source audit checks that z is not returned, aliased, or explicitly stored. A compiled-kernel inspection checks workspace and spill behavior for the exact binary. A device-counter comparison measures transfers under a declared workload and cache state. The first does not imply the other two. Binary hashes bind inspected code to an artifact; they do not prove that a reported remote run actually used it.

During training, autograd may retain controller intermediates and descriptors. The inference no-store contract therefore does not automatically extend to backward. Deterministic rematerialization is one possible training strategy; Section 7.4 tests software checkpointing, not the custom kernel's backward implementation.

4 Representation Limits and a Stronger Control

4.1 Fixed output-subspace obstruction

A conditional core changes coefficients in fixed rank directions. In one projection, the effective matrices belong to the linear span of the r outer products formed by columns of U and rows of V; the span has

dimension at most r. The input-output mapping may still be nonlinear because z depends on x. The final output restriction is stronger and independent of the controller's complexity.

Theorem 1 (Output-subspace obstruction)**.** *Let $V_d \in \mathbb{R}^{r\times d}$ be the fixed final factor in Eq. (5), let $\mathcal{S}$ be its row space, and let $P_{\mathcal{S}}$ be the orthogonal projector onto $\mathcal{S}$. Stack N target FFN outputs into $Y \in \mathbb{R}^{N\times d}$ and corresponding model outputs into $\widehat{Y}$. Then, for any controller,*

$$\|Y - \widehat{Y}\|_F^2 = \|Y(I - P_{\mathcal{S}})\|_F^2 + \|YP_{\mathcal{S}} - \widehat{Y}\|_F^2 \geq \|Y(I - P_{\mathcal{S}})\|_F^2. \tag{8}$$

If the final factor is optimized subject to rank at most r, every candidate still satisfies

$$\|Y - \widehat{Y}\|_F^2 \geq \sum_{j>r} \sigma_j(Y)^2. \tag{9}$$

Proof. Every output row is a coefficient row multiplied by V_d, so $\widehat{Y}P_{\mathcal{S}} = \widehat{Y}$. The two terms in the decomposition $Y - \widehat{Y} = Y(I - P_{\mathcal{S}}) + (YP_{\mathcal{S}} - \widehat{Y})$ have orthogonal row components, giving the Pythagorean identity. Also $\operatorname{rank}(\widehat{Y}) \leq r$; the best rank-$r$ approximation of Y has squared Frobenius error equal to its singular-value tail (Eckart & Young, 1936; Mirsky, 1960). □

The theorem is a structural consequence of classical low-rank approximation, not a new general approximation theorem. It explains a concrete limitation: context-dependent coefficients cannot recover target components orthogonal to the final persistent factor. The lower bound is evaluated on *uncentered* target outputs. For residual-block outputs, subtract the unchanged skip connection before applying it. An arbitrary coefficient matrix can attain the best projection onto a fixed subspace, but the restricted FFN/controller need not attain it. The theorem does not say that controllers are useless within that subspace, nor that all layers of a residual Transformer share one subspace.

A useful future diagnostic reports three quantities on fixed evaluation activations: actual reconstruction error, the fixed-factor floor in Eq. (8), and the best rank-r output floor in Eq. (9). Their gaps distinguish coefficient approximation, subspace selection, and unavoidable rank loss. These real-model floors cannot be recovered from scalar MSE summaries alone and are not reported as measured here.

4.2 An activation-weighted constant-core baseline

A shuffled context control changes the input to the controller and may change the optimization process. It does not by itself show whether a learned dynamic core can be replaced by an efficient constant. A sharper projection-level control fits a constant using training/calibration activations only.

Proposition 1 (Optimal frozen-projection constant)**.** *For fixed row-vector projection inputs $a = xU$, fixed V, and frozen dynamic outputs $z(x)$, write $z(x)$ and c as column vectors in this proposition. Define $Q(x) = \operatorname{diag}(a)VV^\top \operatorname{diag}(a)$, $H = \mathbb{E}[Q(x)]$, and $b = \mathbb{E}[Q(x)z(x)]$. Assume the needed second moments are finite. The constants minimizing*

$$R(c) = \mathbb{E}\big[\|(a \odot (z(x) - c)^\top)V\|_2^2\big] \tag{10}$$

solve $Hc = b$. A minimum-norm solution is $c^* = H^\dagger b$; all other solutions add a vector in $\ker(H)$.

Proof. Q is positive semidefinite and $R(c) = \mathbb{E}[z^\top Qz] - 2c^\top b + c^\top Hc$. If $v \in \ker(H)$ then $\mathbb{E}[v^\top Qv] = 0$, so $Qv = 0$ almost surely and $v^\top b = 0$. Thus b belongs to the range of H, the normal equations are consistent, and completing the quadratic gives the stated minimizers. □

The empirical version replaces expectations by calibration averages; its constant folds into $U \operatorname{diag}(c^*)$. An ordinary mean of z ignores the activation and output metric Q. This is a diagnostic application of weighted least squares, with no claim of novelty for the normal equations. Its optimum is calibration-local, with no automatic held-out guarantee. In the three-stage FFN, replacing earlier cores changes later inputs, so separately fitted projection controls require a new full-FFN and full-model evaluation. No constant-core quality improvement is inferred from the present archive.

4.3 Sensitivity and invariances

Let $A = \phi(XW_g)$, $B = XW_u$, and tildes denote approximations fixed within one group. If ϕ is L_ϕ-Lipschitz, $\|W_p - \widetilde{W}_p\|_F \le \delta_p$, $\|B\|_{\max} \le M_u$, and $\|\widetilde{A}\|_{\max} \le M_g$, then

$$\|Y - \widetilde{Y}\|_F \le \|W_d\|_2 \|X\|_2 (M_u L_\phi \delta_g + M_g \delta_u) + \|\widetilde{A} \odot \widetilde{B}\|_2 \delta_d. \tag{11}$$

To verify it, use $C - \widetilde{C} = (A - \widetilde{A}) \odot B + \widetilde{A} \odot (B - \widetilde{B})$ and $Y - \widetilde{Y} = (C - \widetilde{C})W_d + \widetilde{C}(W_d - \widetilde{W}_d)$, then apply submultiplicativity. Token-conditioned matrices permit pointwise bounds followed by aggregation; a fixed-matrix SVD optimum alone does not establish conditional learnability.

Two paired rank scales are redundant since $s \odot s'$ is a single scale. Removing them does not make the complete factorization identifiable. For invertible diagonal D, $U \mapsto UD$ and $V \mapsto D^{-1}V$ preserve Eq. (1) because diagonal matrices commute. All storage comparisons count actual parameters rather than counting equivalent parameterizations as distinct capabilities.

5 Memory Accounting at a Declared Interface

5.1 Four quantities that must remain separate

Dense parameter cardinality is $P_D = 3dm$, and persistent factor cardinality is $P_F = 3r(d + m)$. A call generates $D_z = 3nr$ diagonal scalars. These are counts of objects; adding them does not produce a lower bound on DRAM traffic. For example, a warm cache can suppress DRAM reads of persistent factors, while multiple tiles can load the same factor repeatedly.

Let b_w, b_G, b_z be bytes per factor, controller parameter, and descriptor scalar. Parameter storage is $b_w P_F + b_G P_G$. Let L_{common} denote transfers across one declared global-memory interface that are shared by two implementations. It includes persistent-factor loads, controller loads, source activations, scaled rank activations, and outputs according to the actual schedule. It is not interchangeable with parameter storage. Physical DRAM bytes require device counters and depend on caching and writeback.

Proposition 2 (Removable final-descriptor transfers)**.** *Suppose two schedules are identical except that one writes and reloads each of the $3nr$ final descriptor scalars exactly once while the other consumes those values locally. With $C_z = 3nrb_z$,*

$$L_{\text{mat}} = L_{\text{common}} + 2C_z, \qquad L_{\text{fused}} = L_{\text{common}}. \tag{12}$$

If the common schedule includes one full cold read of every factor, $L_{\text{common}} \ge 3r(d+m)b_w > 0$, and therefore

$$\frac{L_{\text{mat}}}{L_{\text{fused}}} \le 1 + \frac{2nb_z}{(d+m)b_w}. \tag{13}$$

Proof. The only differing transfers are a store and load of C_z bytes each. Divide $L_{\text{common}} + 2C_z$ by L_{common} and substitute its lower bound. □

This is a scheduled-byte statement, not a physical-HBM or latency bound. Different tiling, cache reuse, launch counts, controller-logit temporaries, casts, or arithmetic violate the matched-schedule premise. In particular, an eager materialized implementation may store more intermediates than the final z; a fused implementation can remove those too. The bound isolates only the final-descriptor round trip. It prevents attributing all improvements from an unrelated multi-operation rewrite to one missing tensor.

The corrected regime-map implementation reports scratch cardinality separately. Its partial no-store global ledger is $e_F b_w P_F + e_G b_G P_G + c_x BSd\, b_x$; materialization adds $2C_z$. Common output and other activation transfers are not included, so these partial totals must not be presented as whole-FFN traffic. The implemented ledger uses $b_z = b_w$; Eq. (13) allows distinct widths. Version 2 explicitly changes the previous mixed-interface totals.

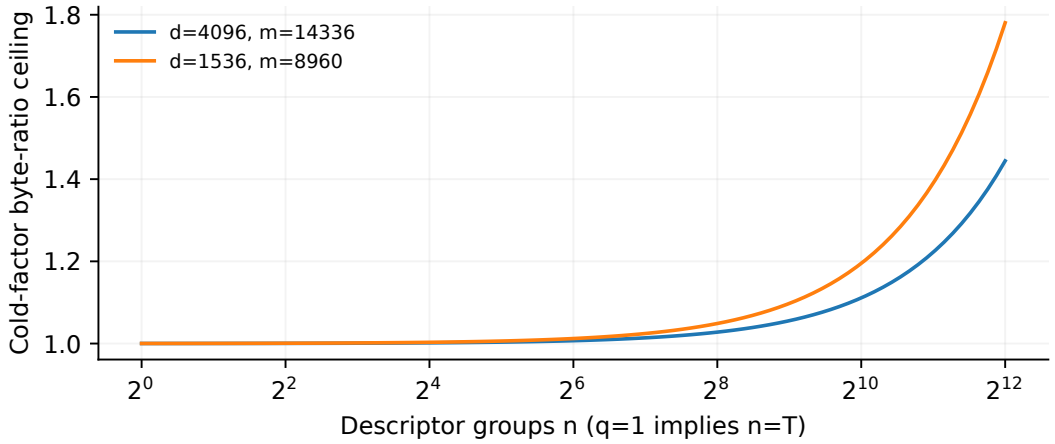


Figure 2: Analytic upper ratio in Eq. (13), assuming equal factor/descriptor widths and one cold full-factor read. This is not measured speedup. For $d + m = 18432$, the ceilings are 1.01389 at $n = 128$ and 1.44444 at $n = 4096$. Rank cancels in this isolated model.

5.2 Full-factor generation and reuse

Generating all factor pairs per group emits $3nr(d+m)$ scalars instead of $3nr$. Relative to the dense parameter cardinality, the one-copy ratio is $n\gamma$, where $\gamma = r(d+m)/(dm)$. For positive integer dimensions and $0 < \gamma < 1$, let $n_{\max} = \lceil 1/\gamma \rceil - 1$ and $g_{\max} = \lfloor n_{\max}/B \rfloor$. Strict cardinality reduction is possible exactly when $g_{\max} \geq 1$, and then it requires

$$q \geq \lceil S/g_{\max} \rceil . \tag{14}$$

Indeed, $B\lceil S/q \rceil \gamma < 1$ is equivalent to $\lceil S/q \rceil \leq g_{\max}$. At $(d, m, r, B, S) = (4096, 14336, 128, 1, 1024)$ this gives $q \geq 43$; at $(1536, 8960, 384, 1, 1024)$ it gives $q \geq 342$. These are optimistic generation-volume thresholds, not traffic or quality claims. The causal grouping restriction still applies.

5.3 Training and routing

Under dense synchronous data parallelism, communicating gradients for fewer persistent parameters can reduce communication volume. The generated values are functions of controller parameters and are not independent optimizer parameters. This explains a low-rank communication saving regardless of whether descriptors are materialized during inference.

A runtime router saves work only when it dispatches the selected branch before that branch's weights and kernels are consumed. Computing both branches and then selecting with `torch.where` provides output selection without the claimed branch-skipping savings. A quality threshold selected on evaluation data also gives an optimistic policy estimate. A future router evaluation must use separate training, calibration, and final evaluation data, fixed routing rules, and whole-call transfer measurements. This paper makes no population tail-risk guarantee or operational router claim.

6 Evidence Protocol and Provenance

This section is written as a reusable protocol, not only as a report of this study. Each element instantiates for a generic conditional low-rank layer: the question set, the matched-contrast invariants of Table 1, the three verification levels, the evidence taxonomy below, and the pre-declared validation gates. An auditor applying the protocol to another architecture substitutes its modulation rule and controller, then reruns the same gates before accepting any efficiency claim. The empirical questions are: (Q1) does conditioning improve reconstruction at a matched budget; (Q2) does fusion improve the same conditional computation; (Q3) can

Table 2: Evidence classes after the October 22, 2026 audit.

Evidence	Available artifacts	Supported scope
Qwen FFN quality	Four per-seed summaries and duplicate aggregate record	Arithmetic and within-archive contrasts; original run provenance incomplete
Four-arm v1 timing	One same-run summary per geometry	Recorded medians; no local per-iteration uncertainty
Multicore v4	Timing/equivalence records; failed stability record superseded by a passing hardened re-run (harness v3, 2026-11-08)	Repeated-call-validated operator speedups over single-core v1; no device counters, no trained-model claim
DDP optimizer timing	Three complete JSON run reports	Short kernel-execution timings; numerical health unknown
CPU checkpointing	New source, nine cases, hashes and tests	Actual-module saved-storage and gradient verification
Algebra checks	New script and 50 seeded fixtures	Numerical consistency of identities; no model-quality evidence

software rematerialization preserve the computation and gradients; and (Q4) do quality and efficiency hold simultaneously in a complete model? Table 2 makes the available evidence explicit.

Archive integrity. The canonical quality family is `asi2_rank512_control_20260922T080042Z`. Its two local files have SHA-256 hashes recorded in the audit. Their per-seed records agree exactly. Hash matching establishes integrity of those files, not independent proof of execution. The canonical directory lacks original per-seed execution reports, trained checkpoints, and complete immutable data manifests. Several alternative closing-run report fragments cannot be parsed as complete JSON and are excluded. The revision preserves historical artifacts and records these limitations.

Quality protocol reported by the archive. The target is the layer-16 FFN of Qwen2.5-1.5B-Instruct, with $d = 1536, m = 8960$, fp32, ranks 384, 401, and 512, 5000 steps, learning rate 2×10^{-4}, batch size one, and 128 evaluation batches. Training seeds are 2026–2029. Factor seed 4242 and activation seed 1729 are shared. Consequently these are four training-seed records from shared data preparation, not four independently disjoint datasets. The record names separate train/test text files and a near-duplicate filter threshold 0.82; the available archive does not establish the exact realized splits or checkpoint identity.

The evaluator reports relative MSE as total squared reconstruction error divided by target squared magnitude for equal-sized evaluation batches. In notation, $\text{RelMSE} = \|\widehat{Y} - Y\|_F^2 / \|Y\|_F^2$ for a nonzero target. It measures FFN output reconstruction, not language-model perplexity or task accuracy. All within-rank contrasts use matching seed labels from one family. The shown intervals are $\overline{\Delta} \pm t_{.975,3} s_\Delta / \sqrt{4}$, with $t_{.975,3} = 3.1824463$. They are unadjusted descriptive intervals under an independent normal paired-run model, whose adequacy cannot be established with four archived summaries. No preregistered equivalence margin, population claim, or multiple-comparison significance claim is made.

7 Controlled Results

Three findings organize this section. First, at the closest matched parameter budget the dynamic arm is *worse*: a 1.95% relative MSE penalty against persistent rank 401 (Section 7.1). Second, the fused dynamic path remains slower than the static alternative it must beat: 3.03× at the archived probe (Section 7.2). Third, the only validated efficiency result is narrower than either headline claim: after the failed v4 stability gate was resolved by the hardened re-run, the multicore kernel sustains 1.05–2.97× lower operator latency than single-core execution at $T = 128$–4096, with 26–66% reduction at $T \geq 512$ (Section 7.2); this is a kernel-latency result without device-counter or trained-model support. No measured result in this section shows dynamic capacity allocation beating a static control at matched budget.

Table 3: Archived layer reconstruction: mean ± sample standard deviation across four training-seed records. Lower MSE is better. Storage includes factors and, when applicable, the controller under the stated $h = 192$ assumption. All rows are recomputed summaries, not newly executed training.

Model	Rank	fp32 MiB	Relative MSE
Static SVD	384	46.125	0.836904 ± 0.001721
Trained persistent	384	46.125	0.449632 ± 0.001417
Shuffled controller	384	48.099	0.449086 ± 0.001302
Token controller	384	48.099	0.448878 ± 0.001323
Static SVD	401	48.167	0.826043 ± 0.001690
Trained persistent	401	48.167	0.440310 ± 0.001374
Shuffled controller	401	50.178	0.439704 ± 0.001242
Token controller	401	50.178	0.439508 ± 0.001273
Static SVD	512	61.500	0.754122 ± 0.001742
Trained persistent	512	61.500	0.383298 ± 0.001197
Shuffled controller	512	63.757	0.381959 ± 0.001184
Token controller	512	63.757	0.381983 ± 0.001191

Table 4: Paired generated-minus-control MSE contrasts. Negative values favor the generated arm. Intervals are descriptive and unadjusted; four records do not certify equivalence, generalization, or independence.

Rank	Control	Mean difference	Descriptive 95% interval
384	Trained persistent	-0.000754	$[-0.001363, -0.000146]$
384	Shuffled controller	-0.000208	$[-0.000261, -0.000154]$
401	Trained persistent	-0.000802	$[-0.001450, -0.000155]$
401	Shuffled controller	-0.000196	$[-0.000257, -0.000134]$
512	Trained persistent	-0.001315	$[-0.001378, -0.001252]$
512	Shuffled controller	$+0.000025$	$[-0.000007, +0.000056]$

7.1 Rank, capacity, and conditioning

Table 3 recomputes every value from the same four seed records. Parameter storage is reconstructed using Eq. (6) and the code-default hidden width $h = 192$; this assumption is explicit because the canonical record omits the resolved width. It is not a measurement of peak memory. A fixed additive 2 MiB controller approximation is inaccurate across ranks and is not used.

At rank 384 the generated-minus-persistent difference is -0.0007545; its descriptive interval excludes zero (Table 4). This numerical finding does not establish a meaningful task-quality improvement. The matched-rank generated-minus-shuffled differences are approximately -0.000208 and -0.000196 at ranks 384 and 401, about 0.05% of the control MSE. At rank 512 the difference is approximately $+0.000025$ with an interval spanning zero. Shuffling is therefore consistent with little observed dependence on the correct token context in these summaries. It does not prove that the learned controller is constant, or predict its behavior on other distributions.

The closest parameter-budget comparison has generated rank 384 at 48.099 MiB and persistent rank 401 at 48.167 MiB. Their mean relative MSE values are 0.448878 and 0.440310: the generated arm has a 1.95% relative penalty. These are near-matched rather than exactly equal budgets. Increasing persistent rank to 512 further reduces MSE, but that comparison consumes a larger budget and cannot isolate conditioning. Figure 3 keeps that capacity cost visible. The observed pattern motivates the constant-core control and output-subspace diagnostics; neither has yet been measured on these trained checkpoints.

7.2 Fusion versus materialization

Table 5 uses only same-run pairs in the shipped v1 summary. The probe has $(d, m, r, h) = (4096, 14336, 128, 128)$, $B = 1, S = 128$, fp16, $\alpha = 0.05$, and seed 1234 on Ascend 910B2. The described

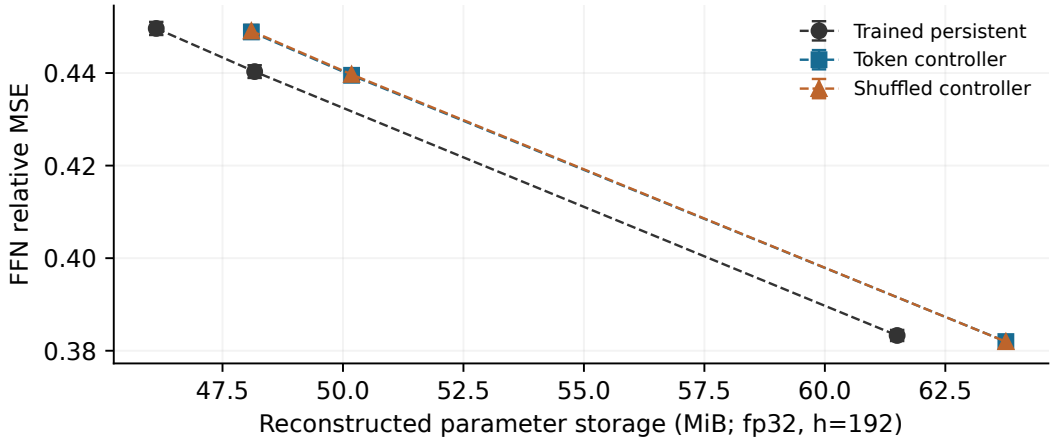


Figure 3: Recomputed archive MSE versus reconstructed parameter storage. Error bars are across-seed sample standard deviations, not corpus-level confidence intervals. Lines join observed rank settings only and do not imply measured interpolation. Persistent low rank is a competitive capacity control.

Table 5: Same-run archived v1 timings in milliseconds. The final column is fused latency divided by materialized latency. The probe includes host-side caching of tiling/workspace and the fp16 controller-weight image.

Tokens	Dense	Persistent	Materialized	Fused v1	Fused/materialized
128	0.4611	0.2719	0.8509	0.8234	0.96768
4096	–	–	1.3932	2.6821	1.92514

protocol uses 30 warmup launches, 200 timed iterations, and five repeats with synchronized host timing. The dense matrices in this harness are exact products of the shared factors; this is an execution control, not an independently trained full-rank checkpoint. The dynamic arms use the same factors and controller, but their outputs need not match the unmodulated persistent arm. Those original full reports are designated remote in the summary; local per-iteration traces are unavailable. Thus the table reports recorded medians without newly asserted statistical uncertainty or cache-residency claims.

At the probe, fusion reduces recorded latency by 3.23% relative to materialization, while it remains 3.03× slower than persistent low rank. At $T = 4096$ it is 1.93× slower than materialization. Both comparisons matter: the first isolates an implementation difference in a generated model; the second asks whether that model is efficient against the practical static baseline. No measured quality-matched deployment improvement follows.

The v4 multicore record was first collected on the original host as an exploratory sweep; its 1000-call stability test ended with `pass=false` and a nonfinite final error, which excluded v4 from validated claims at that time. A 2026-11-08 re-run on a second Ascend 910B2 host with a hardened v3 harness resolves this: after rebuilding the warmup-fixed kernel through the audited CMake/Ascend-C flow (binary verified to embed the verified-warmup probe), the sweep completes 1000/1000 calls with zero transient recoveries and zero persistent failures, final maximum absolute error 0.00390625 against CPU truth, and v1-versus-v4 outputs bit-identical on every probe geometry. The stability gate is therefore marked **PASS** for the warmup-fixed v4 on the tested host and geometry. A matched complete-FFN materialized-versus-fused rerun with a regenerated no-store contract manifest (audited source and binary hashes re-verified against the warmup-fixed build) also passes its equivalence validation (maximum absolute deviation 0.00195 $<$ 0.005 tolerance, fused median 0.894 ms versus materialized 0.869 ms at T=128, d=4096, m=14336, r=128), confirming the fused v1 path is numerically healthy in the same process family. Table 6 reports the validated sweep. Its timing boundary is the `ephemeral_core_apply` custom operator alone, not the complete FFN of Table 5; this is why its v1

Table 6: Validated v1-versus-v4 kernel-latency sweep from the hardened 2026-11-08 re-run, emitted by the audit script from the archived benchmark record. The timing boundary is the `ephemeral_core_apply` operator alone, not the complete FFN of Table 5. Medians of synchronized host timing over the recorded sample; ratio is single-core divided by multicore. The 1000-call stability gate passes on this record (final maximum absolute error 0.00390625, zero transient or persistent failures); this is a kernel-latency result, not a memory-traffic or trained-model claim.

Tokens	v1 single-core op (ms)	v4 multicore op (ms)	Ratio
128	0.1138	0.1087	1.05
512	0.1507	0.1114	1.35
1024	0.2138	0.1241	1.72
2048	0.3099	0.1397	2.22
4096	0.5159	0.1735	2.97

Table 7: Archived optimizer-inclusive synchronized execution, mean ± sample standard deviation across three reports. Ratios use means. These microbenchmarks lack recorded finite-loss/gradient/update health gates, so they support execution-time accounting only.

Arm	Time (ms)	Dense/arm ratio
dense	24.5906 ± 0.5983	1.000
mwg r32	6.0953 ± 0.2943	4.034
mwg r64	6.1529 ± 0.4865	3.997
mwg r96	6.5256 ± 0.9379	3.768
mwg r128	6.0112 ± 0.0996	4.091
mwg r192	5.9397 ± 0.1202	4.140
mwg r256	5.9841 ± 0.2141	4.109
mwg r384	5.9887 ± 0.2008	4.106
mwg r512	7.2919 ± 1.2630	3.372

column (0.114 ms at T=128) is not comparable to the fused-FFN latency there (0.823 ms). At $T \geq 512$ the multicore kernel reduces single-core operator latency by 26–66%. Claim boundaries remain: no parsed HBM/L2 device counters were collected for v4, so the speedup is a repeated-call-validated kernel-latency result rather than a memory-traffic claim, and no joint trained-model quality–latency comparison exists for v4.

7.3 Optimizer-inclusive execution timings

Table 7 recomputes three archived four-device run reports at $d = 4096, m = 14336, B = 2, S = 512$, fp16. The timed region includes zeroing gradients, forward, squared loss, backward, DDP synchronization, and AdamW update. There are three warmup and five timed training iterations per repeat. The reported statistic is the maximum across devices of each device's mean time, then summarized across three repeats. It is not a sample of independent end-to-end training runs to convergence.

The dense mean is 24.5906 ms, and low-rank ratios range from 3.372 to 4.140. The benchmark changes parameter count and controller behavior and does not run the inference descriptor no-store kernel. These ratios cannot identify a lifetime effect. Moreover, the FP16 optimizer benchmark records no finite-loss, finite-gradient, or finite-parameter gate. The revision therefore does not interpret those timings as healthy training, convergence acceleration, or a quality-preserving replacement of an 8B-parameter language model. The phrase "8B-scale geometry" refers only to matrix dimensions.

7.4 Fresh CPU rematerialization verification

To provide complete local evidence for one narrower claim, we executed the actual `LowRankFFN` with `token_diag_core`, comparing ordinary autograd with non-reentrant PyTorch checkpointing. Fixtures use

Table 8: New CPU software measurements, three seeds per depth. Exact byte counts were identical across these seeds. Output, loss, input-gradient and all parameter-gradient maximum absolute differences were zero in all nine cases. The controller was active and all checked tensors were finite.

Depth	Eager saved bytes	Checkpoint saved bytes	Ratio	Cases
1	53,248	4,096	13.0	3
2	106,496	8,192	13.0	3
4	212,992	16,384	13.0	3

synthetic random inputs and randomly initialized factors, $d = 32, m = 64, r = 8, h = 16$, $B = 2, S = 16$, fp32, and $\alpha = 0.2$. Residual stacks $x \mapsto x + \widetilde{f}(x)$ have depths 1, 2, and 4; seeds are 2026–2028. Each arm begins from an identical deep copy. Conditioning response and controller gradients are required to be nonzero.

Saved-tensor hooks count unique underlying nonparameter storages retained by the FFN forward. Aliases of parameters are excluded; shared storage is counted once. Caller-owned inputs and full storage underlying saved views are included. Loss-function saved tensors are outside the measured region. This quantity is not allocator peak memory, total training memory, or hardware traffic.

Python 3.12.14 and PyTorch 2.14.1+cpu were used with one CPU thread. The raw JSON records exact source hashes, initial-state hashes, per-case tensor checks, and the measurement boundary. Tests reject missing or nonfinite gradients and verify parameter-alias exclusion. Table 8 supports software rematerialization for these fixtures. It supplies no evidence about NPU checkpoint speed, peak memory, or trained-model quality. The previously quoted 24.5–71.8-fold memory headline is not carried forward because the available legacy evidence does not establish an instrumented actual-MWG measurement.

7.5 Algebra checks

Fifty seeded float64 NumPy fixtures verify the output-subspace identity and compare the constant-core normal equations with an independently constructed least-squares system. Maximum rowspace residual is 1.42×10^{-14}; maximum relative orthogonal-decomposition residual is 8.88×10^{-16}; constant-core coefficients agree within 3.67×10^{-15}. The fitted constant has no greater calibration loss than the unweighted mean core in these fixtures. These are checks of the derivations, explicitly separated from real-model experiments and from the proofs themselves.

8 Limitations and Decisive Follow-Up Experiments

The theoretical analysis is general within its stated architecture and scheduling assumptions. The available quality evidence is narrow and has incomplete execution provenance. The accelerator evidence comes from one device family, with small repeat counts and different configurations for quality and systems probes. The new CPU experiment establishes a local software property. None of this is an unexpected gap: Q4, a simultaneous full-model quality–latency benefit, is a pre-declared open gate in the protocol of Section 6, and the evidence taxonomy of Table 2 marked it unvalidated before any measurement was recomputed. The audit's contribution is making that status explicit and narrowing what remains: the failed v4 stability gate has been closed by the hardened re-run, so the remaining Q4 preconditions are parsed device-counter evidence and a matched trained-model run on the same artifact.

A decisive next study should freeze a trained causal checkpoint, its tokenizer, dataset splits, and evaluation harness; then measure both quality and runtime for that same artifact. The primary quality comparison should include trained persistent low rank at exact or explicitly near-matched parameter and compute budgets, a calibrated constant-core control, and relevant quantized baselines. Held-out perplexity and task metrics must accompany FFN reconstruction. Rank floors require stored evaluation activations and target outputs; a controller claim should explain the reducible error above those floors.

For execution, the generated/materialized and generated/fused arms must share checkpoint, input, precision, warmup, cache policy, and complete-FFN timing boundaries. Alternate paired launches, report timing

distributions, and verify against an independent high-precision reference before and after extended execution. Gate all timing claims on finite outputs, gradients where relevant, healthy repeated calls, and supported shapes/devices. Record device counters and distinguish logical transfers from DRAM measurements. Test true cached decode ($S = 1$), prefill, varying batch sizes, and the serving stack if a serving claim is made. With stability already resolved (Section 7.2), the remaining precondition for a v4 efficiency comparison is parsed device-counter evidence and a matched trained-model run.

Finally, choose quality margins, primary workloads, repeat counts, and stopping rules before collecting the next comparison. A successful result requires both acceptable held-out quality and an observed efficiency gain on the same implementation. If constant-core or persistent-factor controls dominate, the correct conclusion is a limitation of dynamic modulation in that regime. The mechanism and negative evidence remain useful, but they should not be renamed as an established deployment advantage.

9 Conclusion

This paper delivers an audit methodology for conditional low-rank layers and its application to one architecture, MWG-EW. The methodology keeps representation, storage, and execution claims separate: matched-contrast invariants identify which mechanism a comparison can test, an interface-level ledger assigns each saving to one quantity, and an evidence taxonomy with pre-declared gates grades every measurement. Its application yields a predominantly negative result for dynamic capacity allocation in the studied setting. Ephemeral diagonal cores expose a narrow and testable execution choice within a familiar conditional low-rank model; their fixed final factor imposes an output-subspace floor, and eliminating a descriptor tensor removes only the transfers associated with that tensor under a matched schedule. The archive shows modest same-rank reconstruction differences, a near-budget disadvantage, and no validated complete-model efficiency gain; the one validated efficiency result is narrower, a 26–66% multicore kernel-latency reduction over fused single-core execution at $T \geq 512$ after the v4 stability gate was closed. New actual-module CPU checks establish a reproducible rematerialization property with exact gradient agreement. Both outcomes transfer: the negative findings bound what dynamic-allocation claims this evidence class can support, and the protocol itself can be applied unchanged to the next conditional low-rank architecture before its efficiency is asserted.

Reproducibility and artifact availability

The canonical manuscript, audit script, algebra checks, CPU lifecycle test, and archived source records are in the accompanying research directory. `scripts/build_journal.sh` rebuilds the audit-derived tables, figures, and author-review PDFs. The audit JSON contains exact input hashes and an explicit false submission-readiness flag for the unresolved empirical gates. `paper/REVIEW_20261022.md` maps each review finding to its correction or remaining requirement. Data and checkpoint availability limitations are stated in the evidence protocol, rather than treating a file hash as execution proof.

AI assistance disclosure

Generative AI tools assisted technical review, mathematical derivations, implementation, verification, analysis, and manuscript revision. Newly executed synthetic fixtures are labeled as such; archived accelerator records were reanalyzed without claiming new device execution. The author is responsible for reviewing the arguments, artifacts, and disclosures before any submission.

Broader impact

The intended application is more efficient neural-network execution. Energy savings, production cost reductions, and improved model capabilities have not been measured here. Incomplete numerical checks or overstated performance claims could mislead deployment decisions, which motivates explicit evidence boundaries and preservation of failed gates.

References

Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016. URL https://arxiv.org/abs/1604.06174.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.

Carl Eckart and Gale Young. The approximation of one matrix by another of lower rank. *Psychometrika*, 1: 211–218, 1936. doi: 10.1007/BF02288367.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022. URL https://arxiv.org/abs/2101.03961.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations*, 2023.

David Ha, Andrew Dai, and Quoc V. Le. Hypernetworks. In *International Conference on Learning Representations*, 2017.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

Jaber Jaber and Osama Jaber. Ouroboros: Dynamic weight generation for recursive transformers via input-conditioned LoRA modulation. *arXiv preprint arXiv:2604.02051*, 2026. doi: 10.48550/arXiv.2604.02051. URL https://arxiv.org/abs/2604.02051.

Paras Jain, Ajay Jain, Aniruddha Nrusimha, Amir Gholami, Pieter Abbeel, Kurt Keutzer, Ion Stoica, and Joseph E. Gonzalez. Checkmate: Breaking the memory wall with optimal tensor rematerialization. In *Proceedings of Machine Learning and Systems*, 2020. URL https://arxiv.org/abs/1910.02653.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.

Haokun Liu, Derek Tam, Mohammed Muqeeth, Jay Mohta, Tenghao Huang, Mohit Bansal, and Colin Raffel. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. In *Advances in Neural Information Processing Systems*, 2022. URL https://arxiv.org/abs/2205.05638.

Leon Mirsky. Symmetric gauge functions and unitarily invariant norms. *The Quarterly Journal of Mathematics*, 11(1):50–59, 1960. doi: 10.1093/qmath/11.1.50.

Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron Courville. Film: Visual reasoning with a general conditioning layer. In *AAAI Conference on Artificial Intelligence*, 2018. URL https://arxiv.org/abs/1709.07871.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017. URL https://arxiv.org/abs/1701.06538.

Minshen Zhang, Xiang Hu, Jianguo Li, Wei Wu, and Kewei Tu. Flash multi-head feed-forward network. *arXiv preprint arXiv:2512.06989*, 2025. doi: 10.48550/arXiv.2512.06989. URL https://arxiv.org/abs/2512.06989.